Guía de estudio y exposición

Integración backend + frontend: qué está conectado, cómo funciona y cómo demostrarlo

Proyecto: DigiflAi — monitoreo de flota de UAV

Stack: React + Express + Prisma/PostgreSQL + MongoDB/Mongoose + MQTT

Documento de apoyo para la sustentación

Contenido

[0. Resumen para decir en 30 segundos](#)

[1. Qué está conectado hoy (y qué no)](#)

[2. La idea general: cómo viaja un dato](#)

[3. Diccionario mínimo (con comparaciones que ya conoces)](#)

[4. Por qué usé un seed y no "datos reales"](#)

[5. El recorrido completo de una petición](#)

[6. Qué se agregó al backend para esta integración](#)

[7. Mapa de archivos (dónde mirar)](#)

[8. Cómo levantarlo (sin adivinar)](#)

[9. Guion de la demo (5 a 7 minutos)](#)

[10. Cómo demostrar que NO son datos quemados](#)

[11. Preguntas del jurado y respuestas cortas](#)

[12. Qué decir y qué no decir](#)

[13. Glosario de siglas](#)

[14. Chuleta de comandos](#)

[15. Checklist 10 minutos antes](#)

[16. Si te piden mostrar código, abre estos archivos en este orden](#)

[17. Cierre: trabajo futuro](#)

Guía de estudio y exposición — DigiflAi

> Documento de apoyo para estudiar y presentar el avance de integración backend + frontend. Está escrito asumiendo que sabes Python, Django y PostgreSQL, pero todavía no JWT, CORS, MongoDB ni React avanzado. Todo lo que aparece aquí corresponde al proyecto tal como está hoy.

---

0. Resumen para decir en 30 segundos

> "DigiflAi es una plataforma para monitorear drones. El frontend está hecho en React y el backend en Node.js con Express. Lo que presento hoy es la integración de una parte del frontend con el backend: las vistas de **Panel general** y **Flota UAV** ya dejan de usar datos escritos en el código y consultan la API, que a su vez lee de PostgreSQL y MongoDB. La integración es parcial y funcional: dos vistas completas, el resto sigue con datos de ejemplo. El objetivo era demostrar el flujo completo de un dato, no terminar el producto."

Esa es la idea central. Todo lo demás es detalle para defenderla.

---

1. Qué está conectado hoy (y qué no)

Vista del frontend	¿De dónde salen los datos?	Estado
Panel general (dashboard)	API → PostgreSQL + MongoDB	**Conectada**
Flota UAV (tabla de drones)	API → PostgreSQL + MongoDB	**Conectada**
Telemetría	`src/data/mock.js`	Pendiente
Misiones	`src/data/mock.js`	Pendiente
Alertas	`src/data/mock.js`	Pendiente
Mantenimiento	`src/data/mock.js`	Pendiente
Inteligencia artificial	`src/data/mock.js`	Pendiente
Gemelo digital	`src/data/mock.js`	Pendiente

Palabras importantes para la exposición:

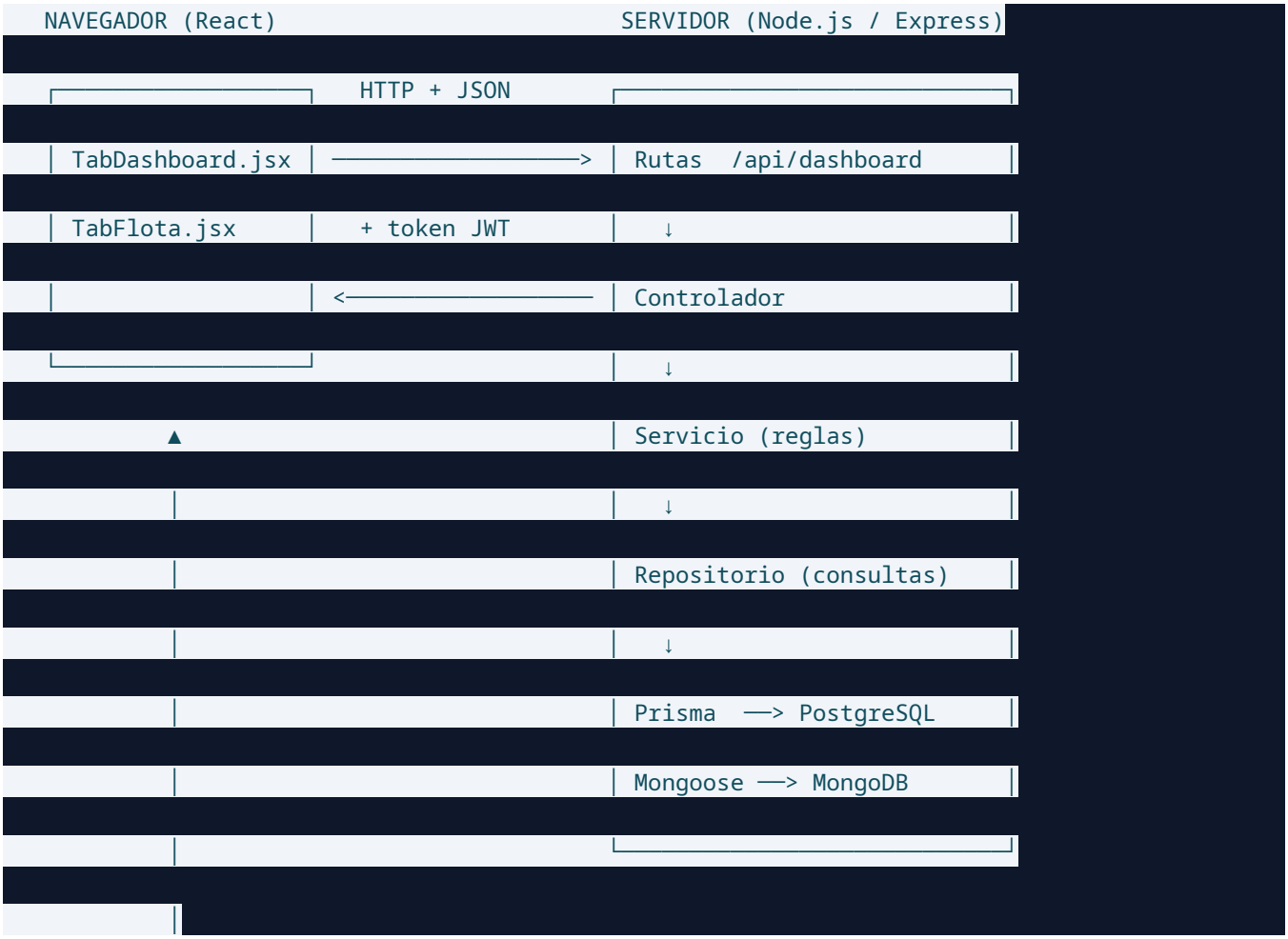
- **"Datos de ejemplo" o "mock"**: información escrita directamente en un archivo de JavaScript. No pasa por la base de datos.
- **"Datos persistidos"**: información que vive en PostgreSQL o MongoDB y se pide por HTTP. Es lo que usan las dos vistas conectadas.

Cantidad de datos que hay hoy en las bases:

Dato	Cantidad	Base
Usuarios	5	PostgreSQL
Drones	6	PostgreSQL
Misiones	4	PostgreSQL
Alertas	4	PostgreSQL
Mantenimientos	3	PostgreSQL
Predicciones de IA	4	PostgreSQL
Lecturas de telemetría	30	MongoDB

---

2. La idea general: cómo viaja un dato



```
      MQTT (EMQX) <── drones / sensores
```

La misma idea que en Django:

Django	Este proyecto
`urls.py`	`src/routes/*.routes.js`
`views.py`	`src/controllers/*.controller.js`
Lógica en servicios	`src/services/*.service.js`
ORM de Django	Prisma
`models.py`	`prisma/schema.prisma`
Migraciones	`prisma/migrations/`
`forms.py` / serializers	`src/validators/*.validator.js`
`manage.py loaddata` (fixtures)	`prisma/seed-demo.js`
Middleware de sesión	`src/middleware/auth.middleware.js`

Si entiendes esa tabla, ya entiendes la arquitectura. El resto son nombres.

---

3. Diccionario mínimo (con comparaciones que ya conoces)

Backend

- **Node.js**: entorno que ejecuta JavaScript fuera del navegador. Es el "Python" de este proyecto.
- **Express**: framework web para Node. Es el equivalente a Django, pero minimalista: tú decides qué middleware y qué rutas existen.
- **Ruta (route)**: una URL asociada a una función. Ejemplo: `GET /api/drones`.
- **Controlador (controller)**: recibe la petición HTTP, llama al servicio y responde JSON. Es como una vista de Django.
- **Servicio (service)**: contiene reglas de negocio ("si el código del dron ya existe, lanza error"). Es código Python normal, sin HTTP.
- **Repositorio (repository)**: solo habla con la base de datos. Separarlo permite cambiar de ORM sin tocar las reglas.

- **Prisma**: el ORM. Tú escribes `prisma.drone.findMany()` y Prisma genera el SQL. Es el equivalente al ORM de Django.
- **PostgreSQL**: base de datos relacional. Tablas, columnas, claves foráneas, enums. Igual que la que ya usas.
- **MongoDB**: base de datos de **documentos**. Guarda objetos JSON completos en lugar de filas. Se usa aquí para telemetría porque llegan muchas lecturas por segundo y no todas tienen los mismos campos.
- **Mongoose**: el "ORM" de MongoDB en Node. Define la forma del documento.
- **MQTT**: protocolo de mensajería ligero para dispositivos. Los drones publican lecturas en un tema (topic) y el backend las recibe. Es como un cartero: el dron deja el mensaje en un buzón y el backend lo recoge.
- **EMQX**: el servidor (broker) MQTT. Está en Docker, puerto 1883.
- **JWT**: un token firmado. Al iniciar sesión, el backend entrega una cadena que el frontend guarda y envía en cada petición. El backend verifica la firma sin consultar la base de datos. Analogía: la manilla de un evento; con ella circulas, y el personal verifica que no sea falsa.
- **CORS**: regla de los navegadores. Si tu página está en `localhost:5173` y llama a `localhost:4000`, el navegador pregunta al backend si ese origen está permitido. El backend responde con `Access-Control-Allow-Origin`. Sin eso, el navegador bloquea la lectura.
- **Joi**: librería de validación. Revisa que el cuerpo de la petición tenga los campos correctos antes de tocar la base de datos. Es como `forms.py` de Django, pero para JSON.
- **Seed**: script que carga datos iniciales o de demostración. Equivale a un fixture de Django. **No es la base de datos**: es el cargador.
- **Migración**: archivo SQL versionado que crea o modifica tablas. Igual que las migraciones de Django.

Frontend

- **React**: librería para construir interfaces con componentes (funciones que devuelven HTML). Piensa en "templates que se actualizan solos".
- **Componente**: una función que devuelve interfaz. `TabFlota` es uno.
- **Hook**: función especial de React que añade comportamiento. `useState` guarda valores; `useEffect` ejecuta código cuando el componente aparece.
- **`fetch`**: función del navegador para hacer peticiones HTTP. Es el `requests` de JavaScript.
- **`localStorage`**: almacenamiento pequeño del navegador. Aquí se guarda el token (`digiflai_token`).

- **Mapper**: función que traduce la respuesta de la API al formato visual. Ejemplo: `AVAILABLE` (base de datos) → `OPERATIVO` (pantalla).
- **Vite**: herramienta que compila React. Equivalente a `collectstatic` + servidor de desarrollo, pero moderno.
- **Nginx**: servidor web que entrega el frontend ya compilado en Docker.

---

4. Por qué usé un seed y no "datos reales"

Esta es la duda más importante, y la respuesta corta es:

> **Los datos sí están en las bases de datos. El seed fue solo la herramienta para meterlos ahí de forma ordenada y repetible.**

Comparación con Django:

```
# Django
python manage.py loaddata demo.json
# Este proyecto
cd backend
npm run seed:demo
```

Ambos hacen lo mismo: leer una definición de datos y escribirla en la base. La diferencia es que aquí el "fixture" es un script de JavaScript.

Qué hace `seed-demo.js`

1. Se conecta a PostgreSQL y MongoDB con las credenciales del `.env`.
2. Crea roles y usuarios (con contraseña encriptada con bcrypt).
3. Borra drones, misiones, alertas, mantenimientos y predicciones previos.
4. Inserta los 6 drones, 4 misiones, 4 alertas, 3 mantenimientos y 4 predicciones.
5. Borra la telemetría anterior en MongoDB e inserta 30 lecturas.
6. Imprime un resumen con las cantidades.

Qué NO hace

- **No** es leído por el frontend.
- **No** se ejecuta cada vez que abres la página.
- **No** reemplaza a la base de datos.
- **No** es "información quemada" en el frontend.

El frontend hace `fetch("http://localhost:4000/api/drones")`; el backend consulta PostgreSQL con Prisma; PostgreSQL devuelve las filas; el backend responde JSON. El archivo `seed-demo.js` no participa en ese recorrido.

¿Por qué no insertar a mano en pgAdmin?

Podrías, pero tendrías que crear 6 drones, 4 misiones, 4 alertas, 3 mantenimientos y 4 predicciones respetando claves foráneas y enums. El seed hace eso en un comando y deja el mismo resultado cada vez. Para una exposición eso es valioso: si algo se daña, lo restauras en segundos.

¿Y entonces qué es "dato real"?

En software hay dos sentidos:

- **Dato persistido**: vive en la base de datos (lo que tenemos).
- **Dato de producción**: lo generan usuarios reales en un sistema en uso.

Lo que cargamos son **datos de demostración persistidos**. Para el jurado, la frase correcta es:

> "Los datos están almacenados en PostgreSQL y MongoDB. Los cargamos con un script de inicialización para poder demostrar el sistema con información coherente."

Evita decir "datos reales" sin aclarar, porque puede interpretarse como "datos de producción". Di **"datos persistidos en las bases"**.

Advertencia práctica

Cada vez que ejecutes `npm run seed:demo`:

- Se borran los drones y todo lo que depende de ellos.
- Se borra toda la telemetría de MongoDB.
- Los usuarios se conservan (se actualizan, no se duplican).
- **Los IDs numéricos de los drones cambian.** El frontend no los usa: busca por `code` (`SD-0002`). Tenlo en cuenta solo si haces pruebas con MQTT usando `droneId`.

---

5. El recorrido completo de una petición

Ejemplo real: la tabla de Flota pide los drones.

Lado backend

1. **`src/app.js`** monta las rutas: `app.use("/api/drones", droneRoutes)`.
2. **`src/routes/drone.routes.js`** define: `router.get("/", auth, controller.findAll)`. - `auth` es un middleware: revisa el header `Authorization: Bearer <token>`, valida la firma del JWT y deja los datos del usuario en `req.user`.
3. **`src/controllers/drone.controller.js`** lee `req.query` (por ejemplo `?limit=50`), llama al servicio y responde: `{ success: true, data: [...] }`.
4. **`src/services/drone.service.js`** aplica reglas y delega en el repositorio.
5. **`src/repositories/drone.repository.js`** ejecuta: `prisma.drone.findMany({ include: { operator: true } })`.
6. **Prisma** genera el SQL y lo envía a **PostgreSQL**.
7. PostgreSQL devuelve filas. Prisma las convierte en objetos JavaScript.
8. Express serializa a JSON y responde HTTP 200.

Lado frontend

1. **`src/views/TabFlota.jsx`** llama a `useApiData(() => api.getDrones(50), [])`.
2. **`src/hooks/useApiData.js`** ejecuta la función al montar el componente y guarda tres estados: `data`, `loading`, `error`.
3. **`src/api/client.js`**: - busca el token en `localStorage`; - si no existe, hace login automático con el usuario demo; - envía `Authorization: Bearer <token>`; - si recibe 401, borra el token, se loguea otra vez y reintenta una vez; - devuelve `body.data`.
4. **`src/api/mappers.js`** traduce los datos: - `AVAILABLE` → `OPERATIVO` - `IN_MISSION` → `EN_MISION` - `MAINTENANCE` → `MANTENIMIENTO` - `OFFLINE` → `FUERA_SERVICIO` - nombre y apellido del operador → un solo texto - `createdAt` → hora `HH:MM:SS`
5. **`TabFlota.jsx`** recorre el arreglo y pinta las filas de la tabla.
6. Mientras llega la respuesta muestra **"Cargando flota desde la base de datos..."**. Si falla, muestra **"Error al consultar la API: ..."**.

¿Por qué hay mappers?

Porque los enums de PostgreSQL están en inglés (`AVAILABLE`) y la interfaz está en español (`OPERATIVO`). En vez de llenar el JSX de `if`, se traduce en un solo archivo.

---

6. Qué se agregó al backend para esta integración

El dashboard y la flota necesitaban predicciones de IA, pero **no existía tabla para guardarlas**. Se agregó:

Capa	Archivo
Modelo	`backend/prisma/schema.prisma` (modelo `Prediction`)
Migración	`backend/prisma/migrations/20261113171037_add_prediction/`
Repositorio	`backend/src/repositories/prediction.repository.js`
Servicio	`backend/src/services/prediction.service.js`
Controlador	`backend/src/controllers/prediction.controller.js`
Rutas	`backend/src/routes/prediction.routes.js`
Montaje	`backend/src/app.js` → `/api/predictions`

Endpoints nuevos:

- `GET /api/predictions`
- `GET /api/predictions/drone/:droneId`

Endpoints agregados para la demostración del stand

La pestaña **Operaciones** necesita listar mantenimientos y reiniciar la demostración sin ejecutar `npm run seed:demo` en medio de la feria:

Capa	Archivo
Reinicio	`backend/src/services/demo.service.js`
Controlador	`backend/src/controllers/demo.controller.js`

Capa	Archivo
Rutas	`backend/src/routes/demo.routes.js`
Montaje	`backend/src/app.js` → `/api/demo`
Listado	`GET /api/maintenance` (repositorio y servicio ya existían)

- `GET /api/maintenance` — lista los mantenimientos con dron y técnico.
- `POST /api/demo/reset` — borra solo los registros `[DEMO]` y restaura los estados iniciales de drones y misiones. Requiere rol ADMIN.

También se hizo configurable el límite de peticiones para que varios visitantes puedan usar el stand sin recibir `429`:

Variable	Valor por defecto	Uso
`RATE_LIMIT_MAX`	100	peticiones a `/api/` cada 15 min
`AUTH_RATE_LIMIT_MAX`	10	intentos de login cada 15 min

En `.env` quedaron en 1000 y 200 para la jornada.

También se agregó el campo `vibration` al modelo de telemetría de Mongo, porque el mock mostraba vibración y no se estaba guardando.

Tres errores que se corrigieron en el camino

Sirven para mostrar que hubo depuración real:

1. **Roles**: el middleware recibía un arreglo y lo comparaba como texto. Se corrigió con `roles.flat()`.
2. **Contraseñas expuestas**: `GET /api/users` devolvía el hash de la contraseña. Ahora se elimina antes de responder.
3. **Telemetría paginada**: el controlador pasaba mal los parámetros (`req.query` completo en lugar de `filters, page, limit`).

---

7. Mapa de archivos (dónde mirar)

`backend/`

```
├── prisma/
│   ├── schema.prisma            # tablas y enums (PostgreSQL)
│   ├── migrations/              # historial SQL
│   └── seed-demo.js             # cargador de datos de demostración
├── src/
│   ├── app.js                   # monta rutas y middlewares
│   ├── server.js                # arranca HTTP, WebSocket y MQTT
│   ├── config/                  # prisma, mongo, swagger, env
│   │   └── swagger.paths.js     # endpoints documentados en Swagger
│   ├── routes/                  # URLs
│   ├── controllers/             # reciben HTTP
│   ├── services/                # reglas
│   ├── repositories/            # consultas a la base
│   ├── middleware/              # auth (JWT), roles, errores
│   ├── validators/              # Joi
│   ├── models/telemetry.model.js# esquema Mongoose
│   ├── mqtt/mqtt.client.js      # recibe lecturas de los drones
│   └── sockets/                 # Socket.IO (tiempo real)
└── .env                         # credenciales locales (no se sube a Git)



frontend/
├── src/
│   ├── api/client.js            # peticiones + token
│   ├── api/mappers.js           # traducción de datos
│   ├── hooks/useApiData.js      # carga, error, cancelación
│   ├── views/TabDashboard.jsx   # Panel general (conectado)
│   ├── views/TabFlota.jsx       # Flota UAV (conectado)
```

```
│   ├── views/TabOperaciones.jsx # Flujos A y C + Reiniciar demo (stand)
│   ├── components/              # ui (badges, KPI, paneles), charts
│   ├── data/mock.js             # datos de ejemplo de las demás vistas
│   └── App.jsx                  # menú y cambio de pestañas
├── Dockerfile                   # build + Nginx
└── nginx.conf
```

---

8. Cómo levantarlo (sin adivinar)

Requisitos: PostgreSQL local en `localhost:5432`, MongoDB y EMQX en Docker.

```
# 1. Infraestructura (MongoDB + EMQX). NO arranques "api" ni "postgres".
docker compose up -d mongodb mqtt

# 2. Backend
cd backend
npm install            # solo la primera vez
npx prisma generate    # solo si cambió el esquema
npx prisma migrate deploy
npm run seed           # roles del sistema
npm run seed:demo      # datos de demostración (opcional pero recomendado)
npm start              # API en http://localhost:4000

# 3. Frontend (en otra terminal)
cd frontend
npm install            # solo la primera vez
docker compose up -d --build --no-deps frontend
# o, sin Docker: npm run dev
```

URLs:

Servicio	URL
Frontend	http://localhost:5173
API	http://localhost:4000
Documentación Swagger	http://localhost:4000/api-docs
Prisma Studio	`cd backend && npx prisma studio`
Consola EMQX	http://localhost:18083

> **Importante:** no uses `docker compose up -d` a secas. Ese comando también arranca un contenedor `api` y un PostgreSQL vacío en el puerto 5433, y el front terminaría hablando con el backend equivocado. Usa los servicios por nombre o `--no-deps`.

Verificación rápida con `curl`:

```
# Login
TOKEN=$(curl -s -X POST http://localhost:4000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@digiflai.com","password":"digiflai123"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# Drones
curl -s http://localhost:4000/api/drones?limit=20 \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool | head -30

# Dashboard
curl -s http://localhost:4000/api/dashboard \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Respuestas esperadas: `success: true` y `data` con los registros.

Para detener todo:

```
cd frontend && docker compose stop frontend

# Ctrl+C en la terminal del backend, o si lo dejaste en segundo plano:

kill $(cat /tmp/opencode/backend.pid) 2>/dev/null
```

---

9. Guion de la demo (5 a 7 minutos)

Ensaya esto dos o tres veces. Ten las pestañas abiertas de antemano.

Modo stand (30–60 segundos)

La aplicación abre directamente en la pestaña **Operaciones** para que el visitante vea el flujo sin navegar. Para cada visitante:

1. **Flujo A — ciclo de misión**: se elige un dron disponible (o se deja el primero) y se pulsa **Asignar misión**. El dron pasa a EN MISIÓN y sube *Misiones activas*. Después, **Finalizar misión** lo devuelve a OPERATIVO.
2. **Flujo C — mantenimiento predictivo**: se muestra la predicción real *Motor delantero izq. — 94 % — CRÍTICO* y se pulsa **Programar mantenimiento**. El dron pasa a MANTENIMIENTO y aparece el registro. Después, **Finalizar mantenimiento** lo devuelve a servicio.
3. **Reiniciar demo**: borra solo los registros `[DEMO]`, restaura los estados iniciales de los drones y misiones y deja la pantalla lista para el siguiente visitante.

Frase corta para decir en el stand:

> "La IA detecta un fallo antes de que ocurra y el sistema ejecuta una acción real: envía el dron a mantenimiento. Todo lo que ve queda guardado en PostgreSQL; este botón lo limpia para el siguiente visitante."

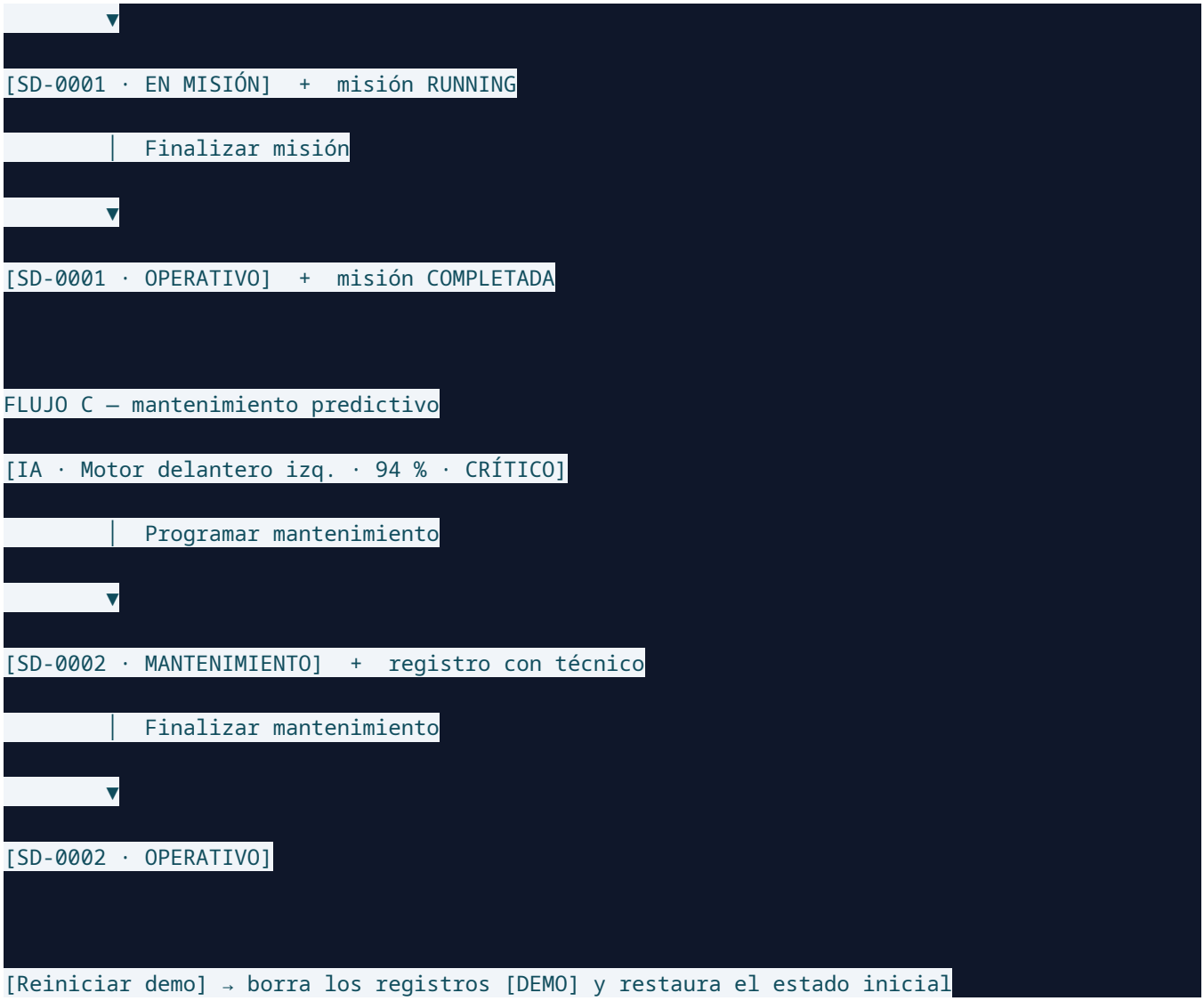
En el stand **no** se ejecuta `npm run seed:demo` (tarda y cambia los ids) ni se abre el login: el cliente ya inicia sesión con el usuario demo.

Diagrama corto para el póster:

```
FLUJO A — ciclo de misión

[SD-0001 · OPERATIVO]

        |  Asignar misión
```



La app siempre usa el **código** del dron (`SD-0001`), no el id numérico, porque el id cambia cada vez que se ejecuta `npm run seed:demo`.

Recorrido largo (5–7 minutos)

Si alguien pide la explicación completa, sigue estos pasos.

Paso 1 — Abrir el frontend (30 s)

Abre http://localhost:5173. La app abre en **Operaciones**; puedes empezar por los dos flujos y después ir a las demás pestañas.

Qué decir:

> "Este es el panel. La navegación y el diseño ya estaban; lo que cambió es el origen de los datos en dos vistas."

Paso 2 — Panel general (1 min)

Señala los cuatro indicadores y el gráfico.

> "Flota activa: 4 de 6. Misiones en curso: 2. Alertas críticas: 1. Riesgo alto: 2. Estos valores no están escritos en el código: los calcula el backend consultando PostgreSQL."

Paso 3 — La prueba técnica (1,5 min)

1. `F12` → **Network**.
2. Recarga.
3. Señala la petición `dashboard`, `drones`, `alerts`, `missions`, `predictions`, `telemetry`.
4. Haz clic en `drones` → **Response**.

> "Aquí se ve el JSON que devuelve el backend. El navegador lo está pidiendo por HTTP; no hay datos quemados."

Si quieres impresionar: abre **Application** → **Local Storage** y muestra `digiflai_token`.

> "El frontend guarda un token JWT y lo envía en el header Authorization de cada petición. El backend valida la firma antes de responder."

Paso 4 — Flota UAV (1 min)

Entra a Flota UAV.

> "Los seis drones con su estado, batería, altitud, velocidad, señal y operador. La batería y la posición vienen de la telemetría almacenada en MongoDB; los datos del dron y del operador, de PostgreSQL."

Paso 5 — Demostrar que es la base de datos (1,5 min)

En otra terminal, marca como atendida la alerta crítica:

```
TOKEN=$(curl -s -X POST http://localhost:4000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@digiflai.com","password":"digiflai123"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")
```

```
curl -s -X PUT http://localhost:4000/api/alerts/1/attend \
  -H "Authorization: Bearer $TOKEN"
```

Recarga el frontend.

> "La alerta crítica bajó a cero porque el backend solo cuenta alertas no atendidas. El cambio quedó en PostgreSQL. Si vuelvo a ejecutar el seed, se restaura."

Este es el momento más fuerte de la demo. Practícalo.

Paso 6 — Swagger (1 min)

Abre http://localhost:4000/api-docs.

1. Entra a **POST /api/auth/login** → *Try it out* → *Execute*.
2. Copia el valor de `token` de la respuesta.
3. Pulsa **Authorize**, pega solo el token y confirma.
4. Prueba **GET /api/dashboard** → *Try it out* → *Execute*.

> "La API está documentada y se puede probar aquí sin escribir código. El login devuelve el token y el botón Authorize lo usa en todas las rutas."

Paso 7 — Cierre (30 s)

> "La integración cubre dos vistas completas. El resto sigue con datos de ejemplo por decisión de alcance. El siguiente paso es el login real, que hoy está simulado, y luego conectar las demás pestañas reutilizando el mismo cliente HTTP."

Plan B si algo falla

- **No carga el frontend**: verifica `docker compose ps` y que diga `healthy`.
- **Error de API en pantalla**: el backend no está corriendo. `cd backend && npm start`.
- **Pantalla vacía o error de red**: revisa que el puerto 4000 responda: `curl http://localhost:4000/health`.
- **No hay datos**: ejecuta `npm run seed:demo` y recarga.
- **Todo falla**: ten capturas de pantalla de respaldo. Una demo con capturas sigue siendo una demo.

---

10. Cómo demostrar que NO son datos quemados

Tres pruebas, de la más simple a la más contundente:

1. **Network del navegador**: hay peticiones HTTP reales y respuestas JSON.
2. **Cambio en la base**: marcar una alerta como atendida cambia el KPI al recargar.
3. **Prisma Studio**: `cd backend && npx prisma studio` abre una interfaz web donde ves las filas reales. Si modificas una batería ahí y recargas el frontend, el cambio aparece.

---

11. Preguntas del jurado y respuestas cortas

¿Qué es un JWT?

> "Un token firmado que el servidor entrega al iniciar sesión. El frontend lo guarda y lo envía en cada petición; el backend verifica la firma. Sirve para no enviar usuario y contraseña en cada llamada."

¿Por qué hay un login automático?

> "Porque todavía no existe pantalla de login. El cliente HTTP inicia sesión con un usuario de demostración y guarda el token. Es temporal y está aislado en un solo archivo; cuando exista el formulario, se reemplaza por una llamada normal."

¿Dónde se guarda el token y es seguro?

> "En `localStorage`. Para una demo es suficiente. En producción se recomienda una cookie `httpOnly` para reducir el riesgo de robo por JavaScript."

¿Por qué dos bases de datos?

> "PostgreSQL guarda datos estructurados y relacionados: usuarios, drones, misiones, alertas. MongoDB guarda telemetría: muchas lecturas por segundo, con campos variables. Separarlas evita que el volumen de telemetría afecte las consultas operativas."

¿Por qué no todo en PostgreSQL?

> "Se podría, pero la telemetría crece muy rápido y no tiene un esquema fijo. MongoDB está pensado para ese patrón. Es una decisión de diseño, no una obligación."

¿Qué es CORS?

> "Una regla del navegador. Si el frontend está en un puerto y la API en otro, el backend debe autorizar explícitamente ese origen. Sin esa autorización el navegador bloquea la respuesta."

¿Qué pasa si la base de datos se cae?

> "El backend arranca igual: `/health` y la documentación responden. Las rutas que consultan datos devuelven error. El frontend muestra un mensaje de error en lugar de una pantalla vacía."

¿Por qué el frontend pide `limit=50`?

> "La API pagina por defecto a 10 resultados. Para la tabla de flota se pide un límite mayor y así se ven todos los drones."

¿Por qué los estados están en inglés en la base y en español en pantalla?

> "Los enums de PostgreSQL usan nombres técnicos (`AVAILABLE`). La interfaz usa español. Un archivo de mappers hace la traducción en un solo lugar."

¿Qué falta para terminar?

> "Tres cosas: pantalla de login real, conectar las demás pestañas reutilizando el mismo cliente, y actualización en tiempo real con Socket.IO. El backend ya emite el evento `telemetry:update`; el frontend todavía no lo escucha."

¿Cómo agregarías otra pestaña?

> "Tres pasos: agregar la función al cliente (`api.getX`), llamarla con `useApiData` en la vista, y mapear la respuesta si los nombres difieren. El patrón ya está resuelto en Flota."

¿Qué rol tiene cada usuario?

> "ADMIN, OPERATOR, TECHNICIAN y RESEARCHER. Solo ADMIN puede gestionar usuarios y generar reportes. OPERATOR y TECHNICIAN pueden operar misiones y mantenimientos."

¿Por qué el seed borra datos?

> "Para que la demo sea reproducible: cada ejecución deja exactamente los mismos registros. No debe usarse en producción."

¿Usaste inteligencia artificial real?

> "No. Las predicciones son datos de demostración guardados en una tabla. La arquitectura ya tiene el espacio para que un modelo las genere y las inserte por la API."

Si no sabes algo

> "Ese punto no lo implementé yo; puedo revisarlo y responderlo con el código en la mano."
>
> Es mejor decir esto que inventar. Un jurado valora la honestidad técnica.

---

12. Qué decir y qué no decir

Di:

- "Integración parcial y funcional."
- "Dos vistas completas, el resto en datos de ejemplo."
- "Los datos están persistidos en PostgreSQL y MongoDB."
- "El frontend consume la API por HTTP con un token JWT."
- "El login es temporal porque aún no hay pantalla de login."

Evita:

- "Está 100% terminado." (No lo está y no lo necesitas.)
- "Son datos reales." (Di "persistidos" o "de demostración".)
- "Se actualiza en vivo." (El frontend no escucha Socket.IO todavía.)
- "Todo el frontend ya está conectado." (Solo dos vistas.)
- "Usé MongoDB para…" si no puedes explicar para qué. Mejor: "la telemetría se guarda en MongoDB porque son muchas lecturas con esquema variable".

---

13. Glosario de siglas

Sigla	Significado
API	Interfaz por la que dos programas se comunican
REST	Estilo de API basado en URLs y verbos HTTP (GET, POST, PUT, DELETE)
JSON	Formato de texto para intercambiar datos
HTTP	Protocolo de comunicación web
JWT	JSON Web Token: credencial firmada
CORS	Cross-Origin Resource Sharing: permiso entre orígenes
ORM	Object-Relational Mapping: traduce objetos a SQL
CRUD	Crear, leer, actualizar, borrar
MQTT	Protocolo de mensajería para dispositivos IoT
UAV	Vehículo aéreo no tripulado (dron)
KPI	Indicador clave de desempeño
Seed	Script de carga de datos iniciales
Mock	Dato simulado escrito en el código

---

14. Chuleta de comandos

```
# Infraestructura

docker compose up -d mongodb mqtt

docker compose ps

docker compose stop mongodb mqtt



# Backend
```

```
cd backend

npm start

npm run seed:demo

npx prisma studio

npx prisma migrate deploy

npx prisma generate



# Frontend

cd frontend

npm run dev

docker compose up -d --build --no-deps frontend



# Verificar API

curl http://localhost:4000/health
```

Credenciales de demostración (todas con contraseña `digiflai123`):

Correo	Rol
admin@digiflai.com	ADMIN
h.moreno@digiflai.com	OPERATOR
c.pinilla@digiflai.com	OPERATOR
j.salas@digiflai.com	TECHNICIAN
a.rua@digiflai.com	TECHNICIAN

---

15. Checklist 10 minutos antes

- ☐ PostgreSQL local encendido.
- ☐ `docker compose ps` muestra `mongodb`, `mqtt` y `frontend` activos.

- ☐ Backend responde: `curl http://localhost:4000/health`.
- ☐ Frontend abre en http://localhost:5173 y muestra datos.
- ☐ La pestaña **Operaciones** abre por defecto y responde a los dos flujos.
- ☐ `Reiniciar demo` deja SD-0001 y SD-0004 operativos, SD-0002 y SD-0005 en misión.
- ☐ `.env` con `RATE_LIMIT_MAX=1000` y backend reiniciado (evita `429`).
- ☐ `npm run seed:demo` ejecutado (para restaurar la alerta crítica).
- ☐ Swagger abre en http://localhost:4000/api-docs.
- ☐ Prisma Studio abierto en otra pestaña (opcional, pero impacta).
- ☐ Terminal con el `curl` de "marcar alerta atendida" listo para pegar.
- ☐ Capturas de pantalla de respaldo.
- ☐ Documento de estudio abierto por si necesitas consultar.

---

16. Si te piden mostrar código, abre estos archivos en este orden

1. **`frontend/src/api/client.js`**

   > "Aquí está el cliente HTTP. Añade el token JWT y maneja el 401."

1. **`frontend/src/hooks/useApiData.js`**

   > "Hook que ejecuta la petición al montar y expone carga y error."

1. **`frontend/src/views/TabFlota.jsx`**

   > "La vista ya no importa `mock.js`; llama a `api.getDrones()`."

1. **`frontend/src/views/TabOperaciones.jsx`**

   > "Aquí están los dos flujos del stand: crear y finalizar una misión, programar y finalizar un mantenimiento, y el botón Reiniciar demo."

1. **`frontend/src/api/mappers.js`**

   > "Traduce los enums de la base al español."

1. **`backend/src/routes/drone.routes.js`**

   > "Ruta protegida por el middleware de autenticación."

1. `backend/src/controllers/drone.controller.js`

   > "Recibe la petición y responde JSON."

1. `backend/src/repositories/drone.repository.js`

   > "Consulta con Prisma, equivalente al ORM de Django."

1. `backend/prisma/schema.prisma`

   > "Modelo de datos: tablas, relaciones y enums."

1. `backend/prisma/seed-demo.js`

   > "Cargador de datos de demostración, como un fixture de Django."

Si solo tienes un minuto, muestra el 1, el 3 y el 8. Cuentan la historia completa: frontend → API → base de datos.

---

17. Cierre: trabajo futuro

Para la diapositiva o frase final:

1. **Login real** con formulario y manejo de sesión.
2. **Conectar las pestañas restantes** reutilizando `client.js`, `useApiData` y `mappers.js`.
3. **Tiempo real** con Socket.IO para que la telemetría se actualice sin recargar.
4. **Pruebas automatizadas** de los endpoints principales.
5. **Despliegue** con variables de entorno y HTTPS.

Frase de cierre:

> "El avance demuestra el flujo completo de un dato: del sensor a la base de datos y de la base de datos a la pantalla. Lo que falta es repetir ese patrón en las demás vistas y agregar el login real."